

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the
BACHELOR DEGREE IN COMPUTER SCIENCE

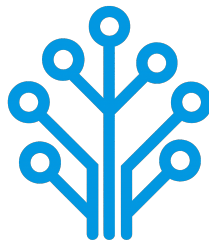
Field of Study : Software Engineering and Information Systems

Design and Implementation of an AI-Based Network Intrusion Detection System

By

AMIRA BOUAFIF & HAJER JEMAA

Conducted within ITXTREE



Publicly defended on June 6, 2026 in front of the jury members:

Chairman:	Name SURNAME, University Relations Leader, IBM
Reporter:	Name SURNAME, Teacher, ISTIC
Examiner:	Name SURNAME, Teacher, ISTIC
Professional Supervisor:	Firas ABBESSI, Engineer, ITXTREE
Academic Supervisor:	Wassim ABBESSI, Teacher, ISTIC

Academic Year: 2025-2026

END-OF-STUDY PROJECT REPORT

Submitted in Partial Fulfillment of the Requirements for the
BACHELOR DEGREE IN COMPUTER SCIENCE

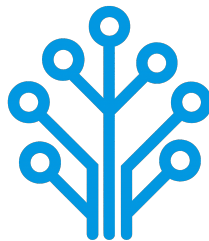
Field of Study : Software Engineering and Information Systems

Design and Implementation of an AI-Based Network Intrusion Detection System

By

AMIRA BOUAFIF & HAJER JEMAA

Conducted within ITXTREE



Authorization of graduation project report submission:

Professional Supervisor:

Academic Supervisor:

Issued on :

Issued on :

Signature:

Signature:

Dedications

I dedicate this End-of-Study project

To my dear parents

Who have supported and encouraged me every step of the way. This is for you, in honor of all the sacrifices you have made for me. I hope to have made you both proud and happy.

To my dear sisters and brothers

Who have been a great support and a source of joy in my life. I am grateful for your love and encouragement throughout this journey.

To all my family and friends

Thank you all for your support and encouragement throughout all this time. I am truly blessed to have you in my life.

Amira BOUAFIF

Dedications

I dedicate this End-of-Study project

To my dear parents

To my dear sisters

To all my family and friends

Hajer JEMAA

Acknowledgments

We would like to extend our sincere gratitude to all those who have supported us throughout the course of this End-of-Study project. Their guidance, encouragement, and support have been invaluable throughout this journey.

First and foremost, we would like to thank our academic supervisor, Professor Wassim ABBESSI, for his guidance and insights, which have been a great help in the successful completion of this project.

Our gratitude also go to ITXTREE and, in particular, our professional supervisor, Mr. Firas ABBESSI, for providing us with the opportunity to work on this project and for their professional mentorship.

We also extend our deep appreciation to the professors at the Higher Institute of Information and Communication Technologies for their invaluable support and teachings, which have greatly contributed to our academic journey.

Finally, we thank everyone who has, in one way or another, helped in making this project a success.

Contents

Dedications	i
Dedications	ii
Acknowledgments	iii
General Introduction	1
1 Project Context	2
1.1 Introduction	2
1.2 Presentation of the company	2
1.3 Study of the existing	2
1.4 Criticism of the existing	3
1.5 Proposed solution	3
1.6 Project Pipeline	4
1.7 Methodologies	4
1.7.1 CRISP-DM	4
1.7.2 UML	5
1.8 Conclusion	6
2 Requirements Specification	7
2.1 Introduction	7
2.2 Functional Requirements	7
2.3 Non-Functional Requirements	7
2.4 Actors Identification	8
2.5 Use Case Diagram	9
2.6 Conclusion	9

3	Conceptual Design	11
3.1	Introduction	11
3.2	Sequence Diagrams	11
3.3	Class Diagram	11
3.4	Deployment Diagram	11
3.5	Conclusion	11
4	Data Collection and Preparation	12
4.1	Introduction	12
4.2	Data Collection	12
4.2.1	Dataset comparative	12
4.2.2	Dataset overview	14
4.3	Data Understanding	14
4.3.1	Data quality	14
4.4	Data preparatation	19
4.4.1	Data cleaning	19
4.4.2	Feature engineering	21
4.4.3	Label encoding	21
4.4.4	Train/ Test split	21
4.5	Conclusion	21
5	Modeling	22
5.1	Introduction	22
5.2	State of the Art	22
5.3	Adopted Approach	22
5.4	Training and Hyperparameter Optimization	22
5.4.1	Initial Setup	22
5.4.2	Baseline training	24
5.4.3	Iterative Experiments	27
5.4.4	Final Model Configuration	33
5.4.5	Hyper parameter tuning	33
5.4.6	Adding thresholds	33
5.4.7	Final model training results	33

5.5	Conclusion	33
6	Realization	34
6.1	Introduction	34
6.2	Development Tools and Technologies	34
6.3	Performance Analysis and Evaluation Metrics	34
6.4	User Interfaces	34
6.5	Integration	34
6.6	Conclusion	34
	General Conclusion	35

List of Figures

1.1	Project Pipeline	4
1.2	CRISP-DM Methodology	5
2.1	Actors	8
2.2	Use Case Diagram	10
4.1	Inf and NaN attack class distribution	15
4.2	Ordering Artifact Class Distribution	16
4.3	Exact duplicates attack class distribution	17
4.4	Contradictory duplicates attack class distribution	18
5.1	Baseline F1-score per class	25
5.2	Web Attack baseline confusion matrix	26
5.3	Bot attack baseline confusion matrix	27
5.4	Infiltration F1-score comparison across configurations	31

List of Tables

2.1	Roles of Actors	9
-----	---------------------------	---

General Introduction

Chapter 1

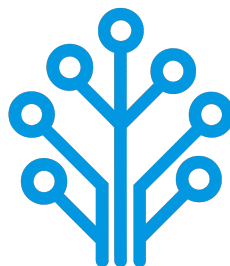
Project Context

1.1 Introduction

This chapter serves as an introduction to the project. We begin with a presentation of the company hosting the internship, a study of existing network intrusion detection systems. Then, we present the problem and the proposed solution. Finally, we end the chapter with a conclusion.

1.2 Presentation of the company

ITXTREE is an early-stage technology startup based in Sidi Rzig, Tunisia, focused on building a solid foundation for future digital solutions it is built on the core values of integrity, quality, curiosity and adaptability.



1.3 Study of the existing

Cybersecurity is a highly sensitive domain where the ability to react quickly can determine the impact of an attack. The earlier a threat is identified, the more effectively its consequences can be limited.

For this reason, Network Intrusion Detection Systems (NIDS) play a central role in monitoring and analyzing network traffic. The most widely used approach relies on:

- **Signature-based:** These systems analyze network traffic, often through deep packet inspection and compares it against a signature database. When a match is found an alert is triggered. Well-known tools such as Snort [1] and Suricata [2] are representative of this approach.
- **Anomaly-based:** Anomaly detection is learning normal behavior of the network through statistical analysis of traffic parameters. If there is a sudden increase of traffic on a port outside of normal thresholds, it might be an indication of a brand-new threat [1].

1.4 Criticism of the existing

Despite their widespread use, signature-based NIDS present several limitations:

- **High Maintenance and Reactivity:** Their effectiveness is essentially tied to the completeness and accuracy of their signature rule sets. As attack techniques continuously evolve, these systems require frequent updates to remain effective. This maintenance process is both time-consuming and reactive since new signatures can only be created after an attack has already been identified and analyzed.
- **Performance:** As the number of rules increases, the system must process a larger set of comparisons for each packet, which can lead to slower detection. In a context where reaction time is critical, this degradation directly affects the system's ability to respond effectively.
- **Rigidity and Evasion Vulnerability:** Attackers can exploit the rigidity of signature-based detection systems by introducing slight variations in attack patterns, allowing them to evade existing signatures while preserving the same malicious intent. Since many signature rules are publicly available or widely shared with the security community, attackers may analyze them to identify gaps and design variants that bypass detection mechanisms.

Overall, these systems rely on exact matching and predefined logics which limits their adaptability in the face of evolving and increasingly sophisticated threats.

1.5 Proposed solution

With the growing integration of Artificial Intelligence (AI) in security systems, new approaches aim to overcome the limits of traditional methods. To address these challenges, we propose an AI-driven NIDS capable of learning patterns from network traffic and classifying different types of network activity instead of relying solely on predefined rules.

1.6 Project Pipeline

A project pipeline is a structured sequence of stages that outlines the workflow and processes involved in the development and deployment of a project. It serves as a roadmap, guiding the team through various phases, from initial planning to final delivery.

Figure 1.1 illustrates the project pipeline for our network intrusion detection system.

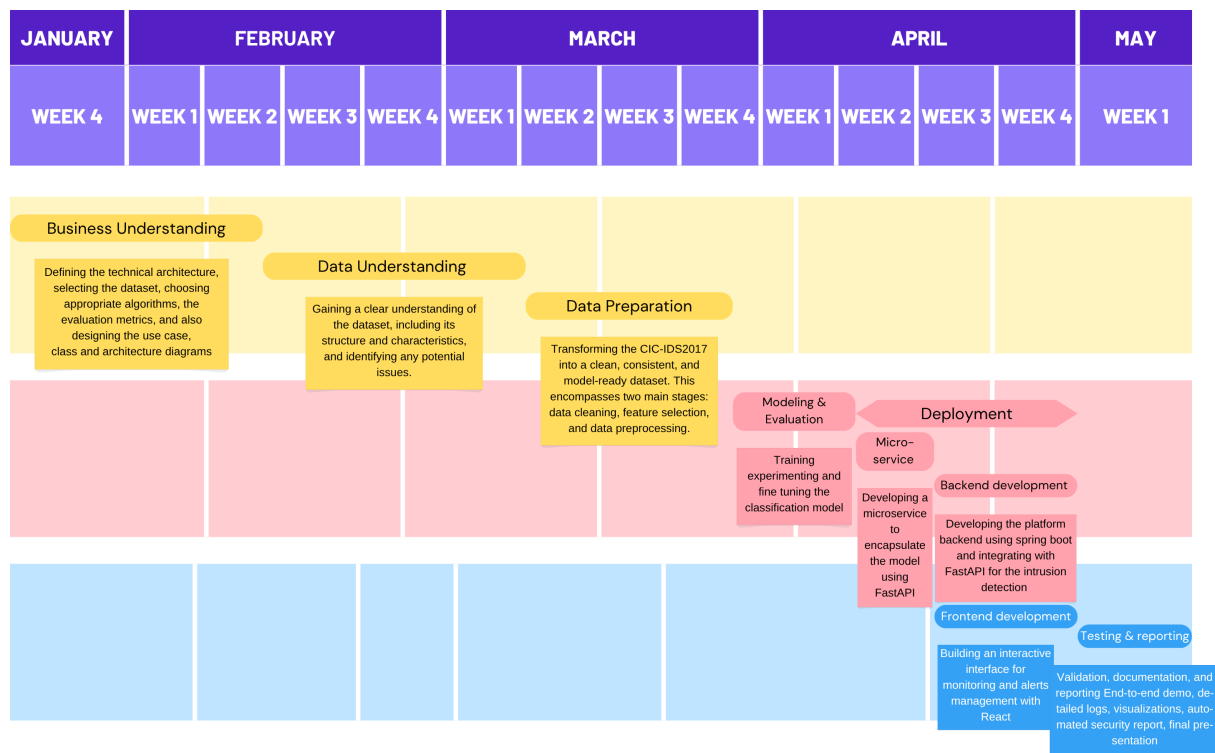


FIGURE 1.1 – Project Pipeline

1.7 Methodologies

In this section, we present the methodologies that we use in this project, including the machine learning methodology CRISP-DM and the software development methodology UML.

1.7.1 CRISP-DM

The CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology is a widely used framework for structuring machine learning projects. It consists of six phases:

- **Business Understanding:** Defining project objectives and requirements and gaining a clear understanding of the business problem.

- **Data Understanding:** Collecting and exploring the data to gain insights and assess its quality by identifying data quality issues.
- **Data Preparation:** Cleaning and preparing the data for modeling by selecting relevant features, handling missing values and splitting the dataset.
- **Modeling:** Selecting and applying appropriate modeling techniques, building, testing and tuning models to optimize their performance.
- **Evaluation:** Evaluating the model's performance and determining if it meets the business objectives.
- **Deployment:** Deploying the model into production, monitoring and maintaining the model's performance.

Figure 1.2 illustrates these phases and their relationships.

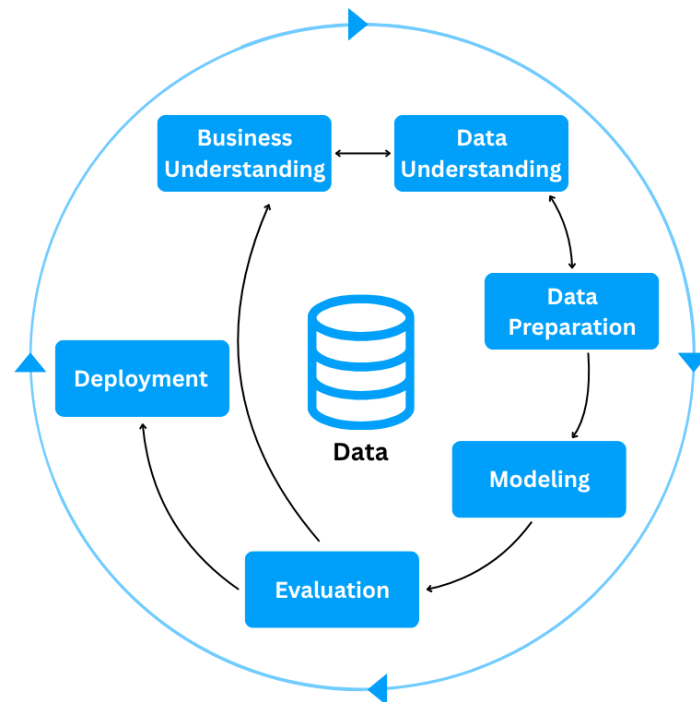


FIGURE 1.2 – CRISP-DM Methodology

1.7.2 UML

For the modeling of the software system, we use UML (Unified Modeling Language), a standardized modeling language used to specify, visualize, construct, and document the artifacts of software systems through diagrams such as use case, class, and sequence diagrams. There are two types of UML diagrams:

- **Behavioral Diagrams:** State machine, activity, use case, sequence, communica-

tion and interaction overview diagrams.

- **Structural Diagrams:** Class, object, component, deployment and package diagrams.

UML makes complex systems easier to design and communicate, it provides a common language for developers, architects, and stakeholders [3].

1.8 Conclusion

In this chapter, we introduce the host company ITXTREE, study existing network intrusion detection systems, discuss their shortcomings and finally present a solution. In the next chapter we specify the system requirements.

Chapter 2

Requirements Specification

2.1 Introduction

Throughout this chapter, we specify both the functional and non-functional requirements, identify the system actors, and present the use case diagram. By defining these elements, we aim to establish a clear and shared understanding of the system's expected behavior and constraints, providing a solid foundation for the subsequent design and implementation phases.

2.2 Functional Requirements

Functional requirements define *what* the system should perform. The main functionalities of our system are as follows:

- Authenticate;
- View Dashboard;
- View Statistics summary;
- Trigger Network Analysis;
- View Alerts List;
- View Reports;

2.3 Non-Functional Requirements

Non-functional requirements define *how* the system should perform its functions. They describe the quality attributes and constraints of the system:

- **Performance:** The system must process network traffic and generate analysis results with minimal latency, ensuring fast detection of potential intrusions.
- **Security:** The system must ensure secure authentication and enforce role-based access control, preventing unauthorized actions and protecting system integrity, while supporting non-repudiation to guarantee accountability and traceability of all operations.
- **Maintainability:** The system should be easy to maintain, allowing for efficient debugging, updates, and modifications to existing components.
- **Usability:** The system should provide an intuitive interface, enabling users to efficiently navigate and interact with its functionalities.

2.4 Actors Identification

Actors are external entities, users, organizations, or external systems that interact with the system to achieve one or more functional requirements, with each actor representing a specific role.

In the context of our project, we identify two primary actors, as illustrated in Figure 2.1

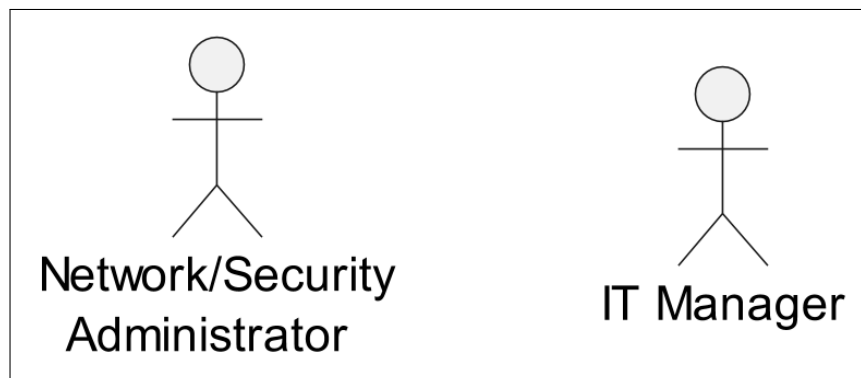


FIGURE 2.1 – Actors

Table 2.1 presents the roles of each actor in our system.

TABLE 2.1 – Roles of Actors

Actor	Role
Network/Security Administrator	- Authenticate - View Dashboard - View Statistics Summary - Trigger Network Analysis - View Alerts List
It Manager	- Authenticate - View Dashboard - View Statistics Summary - View Reports

2.5 Use Case Diagram

A use case diagram is a visual representation of *how* users interact with the system and *what* a system does through different use cases. Figure 2.2 illustrates that diagram.

2.6 Conclusion

In this chapter, we outline the functional and non-functional requirements of our system, identify the primary actors, and represent the way they interact with the system and the system's functionality via a use case diagram. In the next chapter, we present the conceptual design of the platform.



FIGURE 2.2 – Use Case Diagram

Chapter 3

Conceptual Design

3.1 Introduction

3.2 Sequence Diagrams

3.3 Class Diagram

3.4 Deployment Diagram

3.5 Conclusion

Chapter 4

Data Collection and Preparation

4.1 Introduction

4.2 Data Collection

This section introduces the process of acquiring the data used in this project. In general, this phase may involve either constructing a dataset from scratch or relying on existing sources. In our case, the decision to use a public dataset is mainly dictated by the nature of the cybersecurity domain.

Unlike other fields, network intrusion data cannot be collected on demand, as it depends on the occurrence of unpredictable and potentially harmful attacks. In addition, data collection in real operational environments is often constrained due to privacy concerns, security risks, and limited access to organizational traffic. Furthermore, even when such data is available, obtaining reliable ground truth labels is difficult and typically requires extensive manual analysis.

For these reasons, constructing a proprietary dataset is impractical, particularly in the absence of real network traffic provided by the host organization. Consequently, the use of a public dataset becomes a necessary and coherent choice, as such datasets are generated in controlled environments and include labeled instances based on expert-defined attack scenarios.

4.2.1 Dataset comparative

We narrowed it down to these three datasets. Below is a comparative table 4.2.1 using our primary selection criteria.

captionComparison of network intrusion detection datasets

Dataset	Source	Size	Data Generation Approach	Attack Diversity and Temporal Relevance	Usability
NSL-KDD [NSL-KDD10]	Canadian Institute for Cybersecurity at the University of New Brunswick (UNB)	Small ($\approx 148k$ records)	Derived from the KDD '99 dataset, its traffic was generated in a military-style network environment with predefined automated attack scripts.	Low; 4 attack families: DoS, Probe, R2L, U2R with several specific attacks in each family.	High; legacy benchmark widely used in academic studies, mainly for baseline comparison and educational purposes.
UNSW-NB15 [UNSW-NB15]	Australian Centre for Cyber Security (ACCS), UNSW Canberra	Medium ($\approx 2.5M$ records)	Synthetic traffic generated with IXIA PerfectStorm tool.	Medium; 9 attack types: Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms.	High; widely used modern benchmark for machine learning-based intrusion detection research.
CIC-IDS2017 [CIC-IDS2017]	Canadian Institute for Cybersecurity at the University of New Brunswick (UNB)	Medium ($\approx 2.8M$ records)	Traffic generated in an enterprise-like network environment composed of attacker and victim machines, using real attack tools.	High; 14 attack types: Includes the most common attacks based on the 2016 McAfee report, such as Web based, Brute force, DoS, DDoS, Infiltration, Heart-bleed, Bot and Scan.	Very High; one of the most widely adopted and current standard datasets in intrusion detection research.

The comparative analysis of the three datasets reveal that each has its own limitations. NSL-KDD, although still relevant, exhibits limitations in size and temporal relevance, as it is relatively old and does not reflect recent network behavior and modern attacks types. UNSW-NB15 is larger and offers a more recent alternative, but it remains limited in terms of attack type coverage, and its traffic is fully synthetically generated. This makes CIC-IDS2017 the most suitable dataset, as it provides a more realistic experimental setting, a wider range of the most common attack types, and it is also widely adopted in recent research studies.

4.2.2 Dataset overview

CIC-IDS2017 was created by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick (UNB), within a controlled environment, where the network traffic was captured in packet capture (PCAP) format.

The transformation of raw PCAP files was done using a feature extraction tool, CICFlowMeter. Therefore a row in this dataset doesn't represent a packet, it represents an overview of the session behaviour between two endpoints through statistical summary.

4.3 Data Understanding

4.3.1 Data quality

The quality of a model's output is directly tied to its input. In this section we evaluate the dataset for any quality issues that could disrupt model training such as missing values (NaN - Not a number), infinite values (inf), negative values, duplicates rows, constant and near-constant features. For each issue, the analysis investigates the root cause, assesses its global impact on the dataset, across classes and whether it endangers any minority classes.

NaN and Infinite Values: Scanning the dataset for undefined values revealed that NaN values appear exclusively in **Flow Bytes/s**, while infinite values appear in both **Flow Bytes/s** and **Flow Packets/s**. To understand the origin, the CICFlowMeter source code was inspected directly. The relevant formulas are:

$$\text{Flow Bytes/s} = \frac{\text{forwardBytes} + \text{backwardBytes}}{\text{flowDuration}/1,000,000}$$

$$\text{Flow Packets/s} = \frac{\text{packetCount (fwd + bwd)}}{\text{flowDuration}/1,000,000}$$

Both features divide by **Flow Duration**. When **Flow Duration** = 0, any nonzero numerator yields Inf, and a zero numerator yields NaN 0/0. This distinction maps directly onto two observable row types in the dataset:

- Rows with 2 forward packets, 0 backward, and nonzero byte count:
Flow Duration = 0 with bytes > 0, producing **Flow Bytes/s** = Inf.
- Rows with 1 forward packet, 1 backward packet, and zero byte count:
Flow Duration = 0 with zero bytes, producing **Flow Packets/s** = NaN.

The zero byte count in the second case, despite existing packets, is because CICFlowMeter computes **Total Length of Fwd/Bwd Packets** from the packet payload only. Packets carrying no data content only TCP headers (ACK, SYN) produce zero payload length. The true anomaly in both cases is **Flow Duration** = 0.

Flow Duration is computed as the timestamp difference between the last and first packet in a flow. It evaluates to zero when two packets arrive within the same timestamp resolution window as the tool lacks the granularity to distinguish arrivals that are nanoseconds apart. Every affected row contains exactly two packets total, either two forward or one forward and one backward, which is consistent with this explanation: with only two packets, a single resolution failure collapses the entire duration to zero. This collapse propagates to every other feature that depends on the timestamp value, like **Flow IAT Mean**, **Flow IAT Max**, **Flow IAT Min**, and all rate-based features reduce to zero as well.

These rows impact is as follows. Globally, 353 unique row carry at least one NaN value making it 0.01% of the dataset, and 1,564 unique row carry at least one Inf value making it 0.05% of the dataset. At class level, Inf impact BENIGN with 1,427 unique row making it 0.06% of the class samples and only four attack classes are affected. NaN infects only BENIGN with 350 making it less then 0.01% of the class samples along with Dos Hulk. Rare classes such as Heartbleed, Infiltration, and Web Attack – SQL Injection are not affected. The attack class-level distribution is shown in the 4.1 Figure.

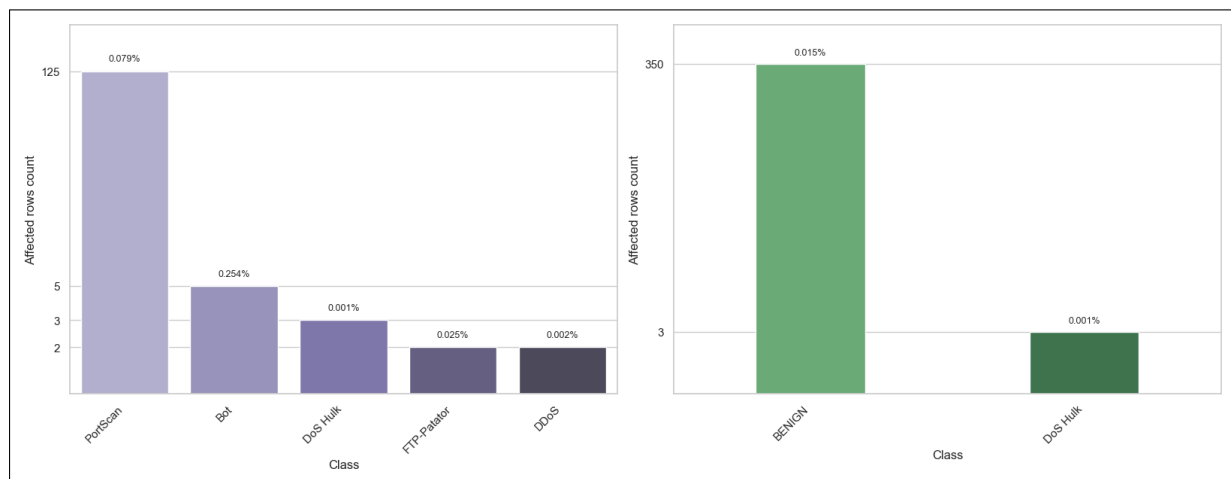


FIGURE 4.1 – Inf and NaN attack class distribution

Negative Values: Given the nature of the dataset feature, their logical value should never be negative yet Negative values were found across several feature groups, each with a distinct cause.

- **Sentinel encoding:** `Init_Win_bytes_forward` and `Init_Win_bytes_backward`.

These two features carry negative values at scale: 1,001,189 rows (35.4%) and 1,441,552 rows (50.9%) respectively hold the value -1. This is not a measurement error. CICFlowMeter uses -1 as a sentinel to signal that the TCP initial window size could not be captured, either because the SYN packet was absent from the capture, or because the flow is UDP-based and has no TCP handshake. It is a deliberate encoding choice by the tool, not a data corruption.

- **Packet ordering artifacts:** `Flow Duration`, `Flow IAT Mean`, `Flow IAT Max`, `Flow IAT Min`, `Fwd IAT Min`, `Flow Bytes/s`, and `Flow Packets/s`.

A total of 115 rows carry negative values in **Flow Duration** ranging from -13 to -1, with the same negative values propagating identically into **Flow IAT Mean**, **Flow IAT Max**, and **Flow IAT Min**.

This is a cascade, since each of these rows has exactly one packet forward and one backward, there is only one inter-arrival interval in the flow, so mean, max, and min all equal **Flow Duration** exactly.

Division by a small negative duration scaled by 1,000,000 then causes **Flow Bytes/s** and **Flow Packets/s** to carry large negative values.

For example, with **Flow Duration** = -1 and 12 bytes of payload:

$$\text{Flow Bytes/s} = \frac{12}{-1/1,000,000} = -12,000,000$$

This is likely to be a packet ordering artifact when processing the packets; additionally, CICFlowMeter performs no guard against negative duration, so the error propagates unchecked. The affected rows are exclusively BENIGN traffic and represent less than 0.0001% of the dataset.

After excluding these 115 rows, an additional 2,776 rows exhibit negative **Flow IAT Min** values and 17 rows exhibit a negative **Fwd IAT Min**. Here the ordering artifact affects only a single packet pair within a longer multi-packet flow, dragging the minimum IAT negative while all other temporal features remain valid. The attack class-level distribution is shown in the 4.2 Figure.

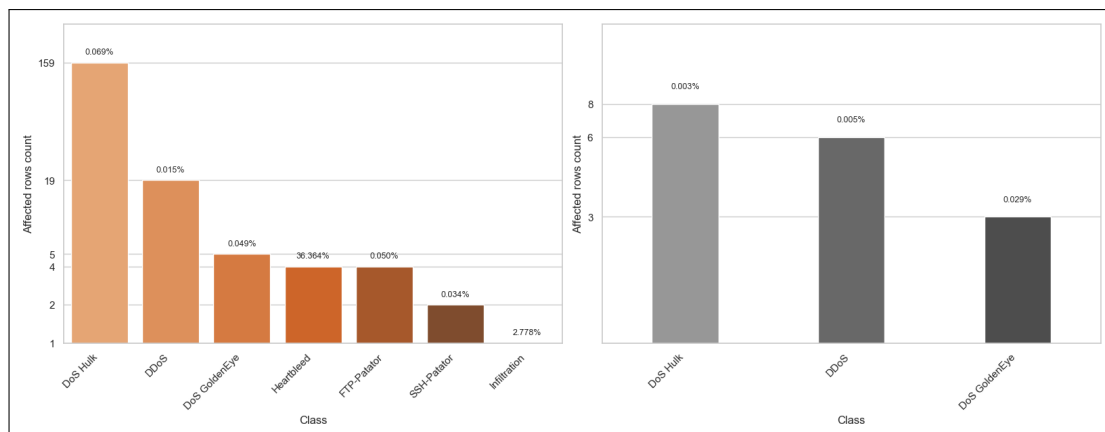


FIGURE 4.2 – Ordering Artifact Class Distribution

- **UDP parsing bug:** **Fwd Header Length**, **Fwd Header Length.1**, **Bwd Header Length**, **min_seg_size_forward**.

These features exhibit values in the range of -32,212,234,632 to -1,073,741,320, far outside any legitimate segment size and a TCP header size, which is bounded between 20 and 60 bytes.

The root cause is a bug in CICFlowMeter: when processing UDP packets, the tool reads the header size from a TCP-specific field that simply does not apply to UDP, producing meaningless values that corrupt these features.

The bug is confirmed by the data: all 35 affected rows are UDP flows, as indicated by **Init_Win_bytes_forward** = **Init_Win_bytes_backward** = -1.

The impact is narrow, 35 rows in `Fwd Header Length`, `Fwd Header Length.1`, and `min_seg_size_forward`, and 22 rows in `Bwd Header Length`, all exclusively BENIGN traffic, collectively under 0.0001% of the dataset.

Exact duplicates: Duplicate rows are common in network datasets since a row in the dataset does not represent a packet but a session behaviour summarised through statistical aggregation. Many flows share identical statistical profiles, thus the presence of duplicates does not necessarily indicate a data error.

Globally, duplicate rows represent 308,381 out of 2,830,743 rows (10.89%). Their distribution by attack class is as show in the 4.3 figure and no minority class is affected.

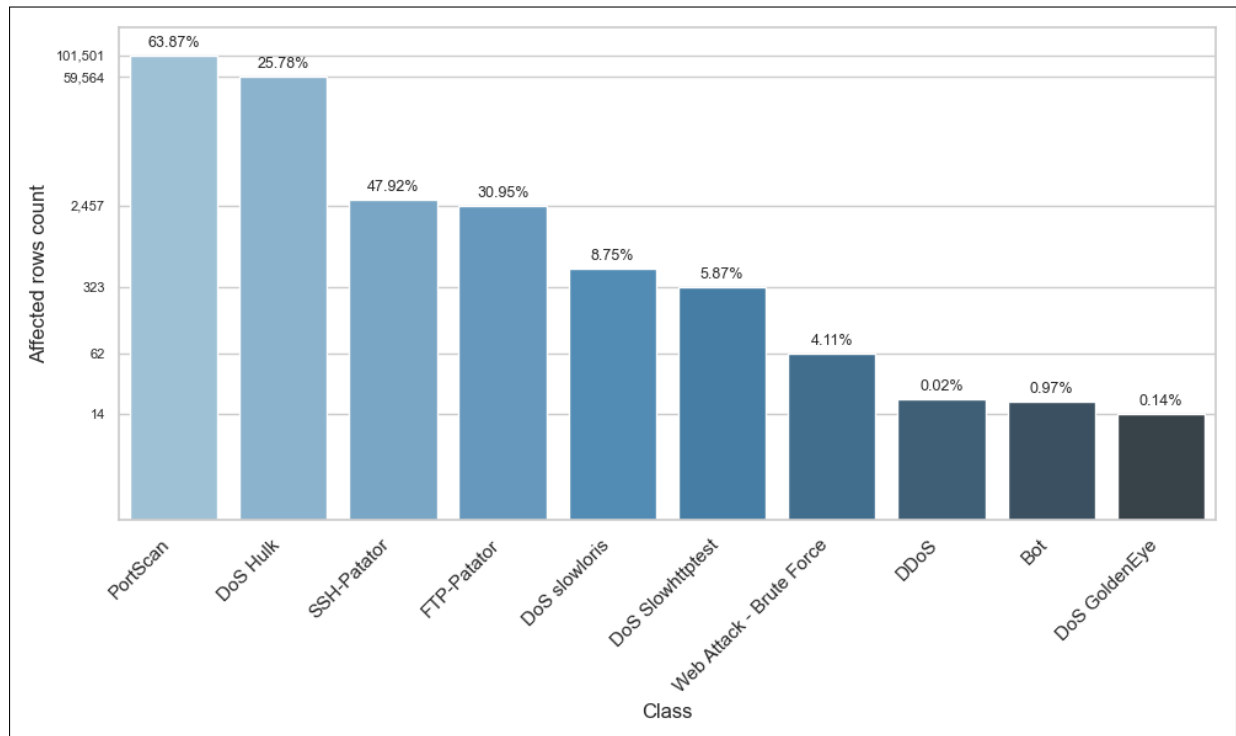


FIGURE 4.3 – Exact duplicates attack class distribution

Contradictory duplicates: These rows share the exact same feature values except for the target column, their labels differ. CICFlowMeter aggregates raw packet-level captures into flow statistics. Two flows from different sessions, one BENIGN and one malicious, can produce statistically identical feature vectors if their traffic patterns are indistinguishable at the flow-aggregation level. This is a fundamental limitation of statistical flow-based features rather than a labelling error.

Globally, 7,020 rows are affected. BENIGN traffic is the dominant participant in contradictory groups, as expected, BENIGN flows vastly outnumber attack flows, so the probability of a BENIGN flow sharing a feature vector with an attack flow is higher than inter-attack collisions. Contradictions involve only well-represented classes: DoS Hulk, DDoS, DoS slowloris, and PortScan. The 4.4 figure illustrates their distribution.

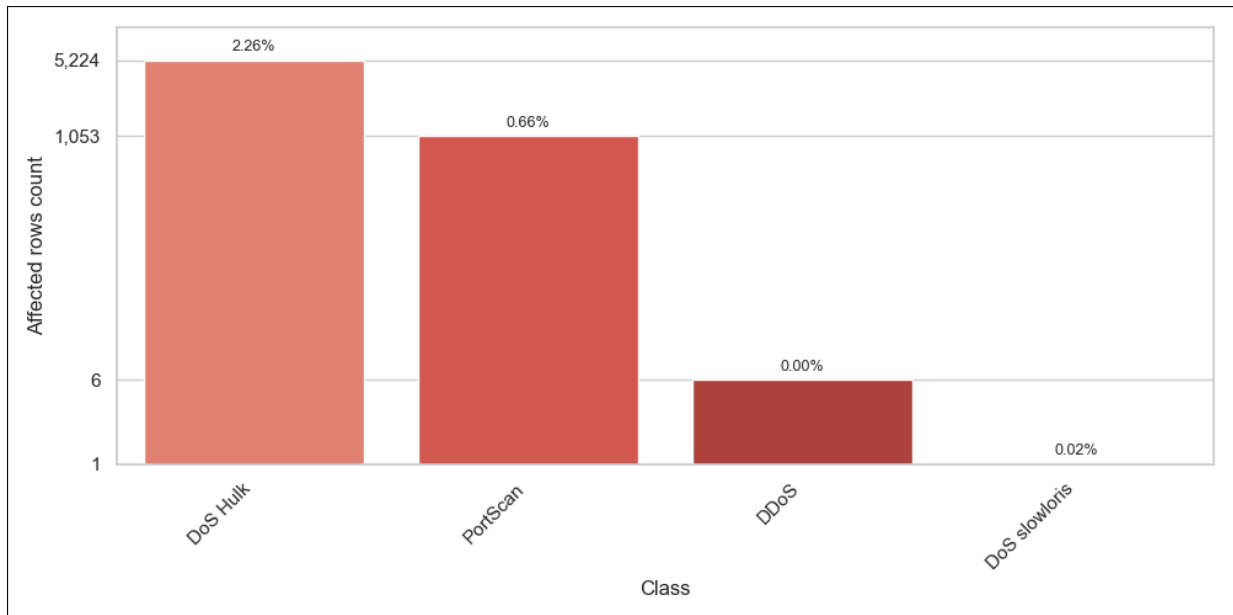


FIGURE 4.4 – Contradictory duplicates attack class distribution

Constant features: Constant features are those with a single unique value across the entire dataset, in this case, always zero. To understand this behaviour, each feature was traced back to its computation logic in the CICFlowMeter source code. The root cause divides them into two groups.

- **Bulk transfer features:** Fwd Avg Bytes/Bulk, Fwd Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, Bwd Avg Bulk Rate.

Bulk transfer refers to sending a large number of packets consecutively in one direction, with very little time between them.

These features are derived from CICFlowMeter’s bulk detection logic, which only activates when at least four consecutive packets carrying actual data are observed in close succession.

If that condition is never met, either because flows in this dataset rarely involve this kind of continuous data sending, or because many packets carry no payload, the counters stay at zero and all six features are exported as zeros.

This is exactly what is observed. These features are constant not by error, but because the bulk detection logic simply never triggers on this dataset.

- **Backward flag counters:** Bwd PSH Flags and Bwd URG Flags

These are constant zero due to a bug in CICFlowMeter’s implementation. The features Bwd PSH Flags and Bwd URG Flags are meant to count how many times the PSH and URG flags appear in packets traveling in the backward direction.

However, these counters are only incremented inside the method that processes the first packet. Since the first packet by definition can only be forward, CICFlowMeter assigns the source IP from that first packet and compares subsequent packets against it to determine direction. The method that handles the following packets never

updates these counters, so both features remain zero for every flow, regardless of the actual traffic.

Near-constant features: Fwd URG Flags, CWE Flag Count, RST Flag Count, and ECE Flag Count.

Each feature hold exactly two unique values across the entire dataset. These are rare-event TCP control flags that are virtually absent in normal traffic. Combined, their non-zero occurrences account for only 315 rows (0.0139%), all within BENIGN traffic. Therefore, they carry no discriminative signal for any attack class.

4.4 Data preparation

4.4.1 Data cleaning

The integrity of data is a prerequisite for any reliable model. Throughout this section, data correctness is prioritized over sample retention, as long as data volume and class balance are not meaningfully affected.

NaN and Inf Values: Rows carrying NaN or Inf values in **Flow Bytes/s** and **Flow Packets/s** are removed. As established, these arise from division by a zero **Flow Duration**, due to the lack of time granularity. Beyond the invalid rate features, these rows are also dominated by zeros which do not reflect the true nature of the flow, but are simply an artifact of insufficient measurement precision. Such a row cannot serve as a valid representation of a network session, as its defining characteristics were lost in the granularity gap.

The scale reinforces the decision. Globally, fewer than 0.11% of rows are affected. Additionally, the unique affected rows per attack class amount to 5 for Bot, 125 for PortScan, 2 for DDoS, and 3 for DoS Hulk, representing less than 0.00% of any single class, all of which carry sufficient clean samples to generalise from. No minority class is affected.

Negative Values: Negative values across the dataset fall into four distinct categories, each handled differently.

- **Sentinel encoding:** `Init_Win_bytes_forward` and `Init_Win_bytes_backward`.

These are retained without modification. As noted, -1 is a deliberate sentinel used by CICFlowMeter to encode the absence of a TCP handshake, it is semantically valid and carries information about the flow type. Replacing it would discard a meaningful signal.

- **Packet ordering artifacts:** This anomaly originates during packet processing, since these sessions consist of a single packet exchange, the flow has no other packets to establish a valid duration or inter-arrival times. **Flow Duration** was rendered

negative, and all rate-based features, `Flow IAT Mean`, `Flow IAT Max`, and `Flow IAT Min` collapsed to that same negative value, leaving no part of the flow intact.

Beyond the specific features where the corruption is visible, there is a deeper concern. This is a bug introduced during dataset construction, and its effects are not fully observable from the CSV alone.

Negative values in `Flow Duration` make the anomaly detectable, but there is no guarantee that the ordering artifact confined its damage only to features that produce obviously invalid numbers.

The same error may have silently distorted other features into values that appear plausible but are incorrect, damage that is invisible precisely because it does not manifest as negatives or zeros, but as ordinary-looking numbers that simply do not reflect the true flow.

In order to preserve data integrity and given the rows effect is negligible, only 115 rows belonging to BENIGN traffic, we decide to drop these rows.

`Flow IAT Min` and `Fwd IAT Min` share the same root cause, a packet ordering artifact, but the outcome is fundamentally different. Here the affected flows span multiple packets, so the ordering error disturbed only a single packet pair within an otherwise well-measured session.

The negative value appears only in the minimum IAT statistic, while `Flow Duration` remains positive and all other features are valid.

Inter-arrival time cannot be negative by definition, so clipping both features to zero restores the value to the physical boundary it should never have crossed, without fabricating anything.

This targeted correction is justified precisely because the flow itself was captured correctly, unlike the single-packet cases above, where the corruption was total and removal was the only option.

This treatment also preserves the 4 Heartbleed rows that would otherwise be discarded, a meaningful consideration given the class contains only 11 samples in total.

- **UDP parsing bug:** `Fwd Header Length`, `Bwd Header Length`, `Fwd Header Length.1`, and `min_seg.size_forward`.

These rows are removed. As established, they are caused by a parsing bug rendering these feature values meaningless and irrecoverable. The fact that all affected rows belong exclusively to BENIGN traffic and represent under 0.002% of the dataset confirms that removal incurs no meaningful cost to class balance or dataset integrity.

Exact duplicates: Exact duplicate rows are dropped. Retaining them risks data leakage, a row could appear in both the training and test sets simultaneously, moreover it risks overfitting by biasing the model toward a single flow pattern. Since no minority classes are affected, the removal carries no risk to rare attack samples.

Contradictory duplicates: These rows present a serious risk to model training. An identical feature vector mapped to more than one target is unresolvable, retaining such rows forces the model to fit contradictory labels from the same input. All rows belonging to contradictory groups are therefore dropped entirely, regardless of their label. Since rare classes are unaffected, this carries no risk of losing minority attack samples.

4.4.2 Feature engineering

4.4.3 Label encoding

The selected model, XGBoost, requires numerical input for both features and the target variable. While all features are already numerical, the target column `Label` is categorical and stored as an object type. Hence, Label Encoding was performed using Label encoder from `sklearn.preprocessing`. The encoder was fitted and applied to the target column in a single step using `fit_transform()`. To preserve interpretability of the model's predicted outputs, the resulting integer-to-class mapping was exported as a `label_mapping.json` file derived from `le.classes_`, allowing any predicted label to be traced back to its original class name.

4.4.4 Train/ Test split

Several factors make us approach this step with caution, when assessing the train/test split strategies.

A per-file split is not an option, as the files are divided per attack type, each attack class appears in exactly one file, so splitting by file would result in classes entirely absent from either the training or test set.

A standard random 80/20 split presents its own issues. The CIC-IDS2017 dataset is heavily imbalanced, the BENIGN to attack ratio is 80/20. Additionally, rare classes present less than 0.001.

For these reasons, a stratified train/test split is selected. It ensures all classes are present in both the training and test sets, each maintaining their original proportions, so the class distribution in each split mirrors that of the full dataset.

However, this strategy doesn't solve the class imbalance. This issue will be addressed and treated more during the model training.

4.5 Conclusion

Chapter 5

Modeling

5.1 Introduction

5.2 State of the Art

5.3 Adopted Approach

5.4 Training and Hyperparameter Optimization

This section details the full training and optimization process of the XGBoost classifier on the CIC-IDS2017 dataset.

5.4.1 Initial Setup

Metric selection :

The CIC-IDS2017 dataset is inherently imbalanced. BENIGN traffic constitutes roughly 80% of the dataset, reflecting the reality of real network traffic where attacks are rare events. Within the attack classes themselves, representation varies significantly, some classes contain hundreds of thousands of samples while others hold fewer than a dozen. This makes metric selection a non-trivial decision that directly affects how honestly model performance is reported.

Accuracy is an intuitive metric but an unreliable one here. It counts correct predictions across all samples divided by the total:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

The sheer volume of BENIGN instances inflates the score regardless of attack detection quality. A model that predicts BENIGN for every single sample would still achieve over

99% accuracy while detecting zero attacks. This makes accuracy effectively blind to what actually matters.

Precision and recall offer more granular insight into model behavior. Precision measures how many of the model’s predictions for a given class are actually correct, and Recall measures how many of the true instances of that class the model successfully detected:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Both metrics are informative individually but insufficient alone; a model biased toward conservative predictions will report high precision at the cost of missing many attacks, while an overly permissive model will recover most attacks but introduce a high rate of false positives.

The F1-score addresses this by taking the harmonic mean of precision and recall, penalizing models that sacrifice one for the other:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

However, standard F1-score or its weighted variant still allows dominant classes to overshadow rare ones in the aggregate score, since each class’s contribution to the final score is proportional to its sample count.

We therefore adopt the Macro F1-score as the primary evaluation metric, which computes F1-score separately for each class, then averages those scores giving every class the same contribution regardless of how many samples it contains. It is defined as:

$$\text{Macro F1} = \frac{1}{C} \sum_{i=1}^C \text{F1}_i$$

where C is the total number of classes and F1_i is the F1-score for class i .

This guarantees that every class, regardless of size, contributes equally to the final score. Per-class F1-score scores are also tracked alongside it throughout all experiments to monitor individual class behavior.

Cross-Validation :

Given the challenges associated with the CIC-IDS2017 dataset, particularly the class imbalance issue and the limited number of samples in certain attack classes, we decide to adopt an experimental approach during the training phase and rely on evaluation metrics to guide our decisions for the final model configuration. This led us to use cross validation.

Cross-validation is an evaluation technique that gives a more stable and reliable estimate of model performance, instead of relying on a single train-test split, the dataset is divided into several folds, and the model is trained and evaluated multiple times, each time using a different fold as the validation set while the remaining folds are used for training. The final performance is then obtained by averaging the results across all runs. Several variants exist:

- Standard k-fold divides the data into k equal folds and rotates the validation fold across all k iterations. It is more practical, but it assigns samples to folds randomly, with no guarantee that rare classes appear in every fold. With classes as scarce as Heartbleed, a random split could place all its samples in the training folds and leave none for validation, or the reverse, making the evaluation meaningless for that class.
- Stratified k-fold addresses this directly. It applies the same k-fold rotation but enforces that each fold mirrors the class distribution of the full training set. If Heartbleed represents 0.0005% we set $k = 5$. This choice is driven primarily by the scarcity of our rarest class: with only 9 Heartbleed samples in the training set, each fold sees approximately 2 instances. Increasing k would reduce this further, making per-fold evaluation for rare classes even less stable. Decreasing it would produce fewer evaluation rounds and increase variance in the aggregated score. Five folds represent a practical balance between evaluation stability and computational cost given this constraint.

Across all experiments, the reported performance corresponds to the mean Macro F1-score over the five folds, with per-class F1-score scores tracked individually to monitor class-level behavior throughout.

Initial XGBoost parameters

5.4.2 Baseline training

A baseline model should be the first model built before moving toward more complex approaches, as it acts as a reference point for evaluating the performance of future configurations.

If a more advanced configuration fails to outperform the baseline or produces even lower results, this may indicate that the additional complexity provides little practical benefit. In such cases, the issue may not originate from the model itself, but may instead suggest potential limitations related to the dataset.

Furthermore, baseline results provide guidance for future steps. For example, if a classification model fails to predict certain classes, efforts should be placed on addressing this flaw.

For these reasons, we establish a baseline model by training XGBoost. The baseline training results divide into expected and unexpected outcomes, as shown in Figure 5.1:

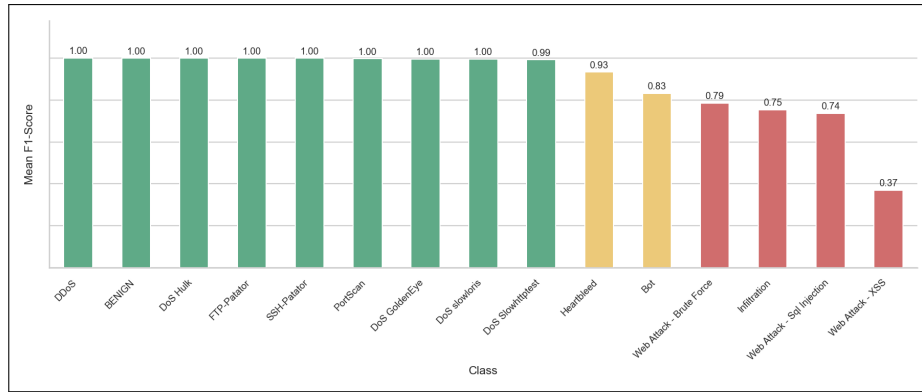


FIGURE 5.1 – Baseline F1-score per class

Expected outcomes :

Classes with large sample counts perform exactly as anticipated. Attack types such as DoS Hulk, PortScan, and DDoS, each with over 100,000 training samples, along with moderately represented classes like DoS GoldenEye, FTP-Patator, SSH-Patator, DoS Slowloris, and DoS Slowhttptest, each above 5,000 samples, achieve a high F1-score at 0.99. The model has enough examples to learn stable, discriminative boundaries for these classes.

At the other end of the spectrum, Heartbleed and Infiltration show lower performance, with F1-score at 0.93 and 0.75 respectively. With only 29 and 9 training samples remaining after preprocessing, this outcome is fully expected, there is simply not enough signal for the model to generalize reliably.

Unexpected outcomes :

- **Web Attacks:** The more revealing result comes from the Web attack subclasses. We anticipated that Web Attack – SQL Injection would struggle, given only 19 training samples. What was unexpected is that all three Web Attack subclasses perform poorly, Brute Force achieves F1-score = 0.78, SQL Injection F1-score = 0.74, and XSS a striking F1-score = 0.37, despite having 522 training samples, supposedly sufficient for a tree-based model to learn from.

The confusion matrix is computed to identify the specific misclassification patterns. As shown in figure 5.2, there is a consistent mutual confusion between XSS and Brute Force, XSS is misclassified 345 times as Web Attack – Brute Force, and Web Attack – Brute Force is misclassified as XSS 167 times. Both are also occasionally classified as BENIGN. This indicates that the model fails to establish meaningful separability between these categories in the feature space.

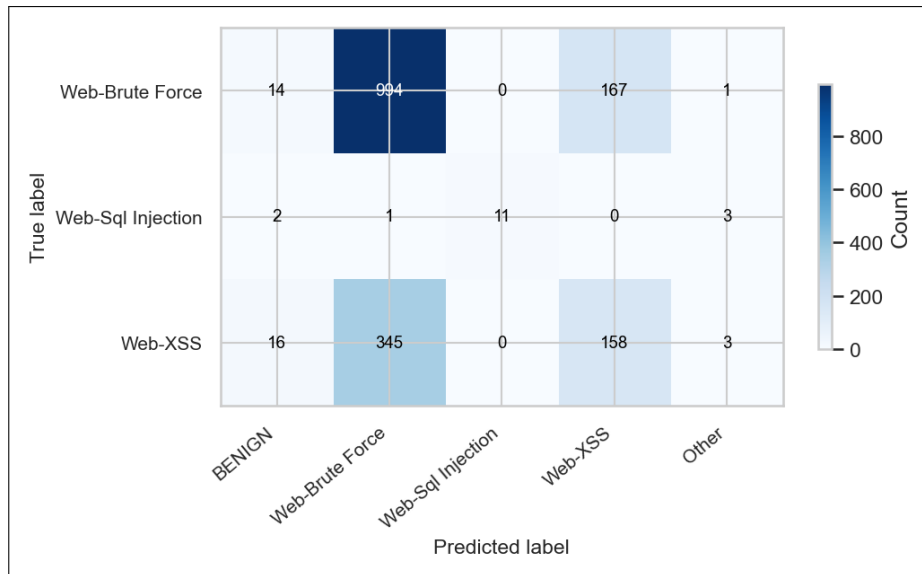


FIGURE 5.2 – Web Attack baseline confusion matrix

Although the Web Attack subclasses remain relatively underrepresented compared to dominant classes in the dataset their consistently poor performance suggests something potentially more fundamental than class imbalance or sample scarcity alone.

Web attacks are payload-based attacks, an XSS carries a malicious script in a HTTP request body, an SQL injection crafts SQL commands by tempering with input fields interfering with database queries. Flow-based datasets like CIC-IDS2017 record features derived from packet timing, length distributions, and header-level metadata; there is no payload field. These features are evidently insufficient to capture the semantic structure of such attacks.

- **Bot attack:** Bot is still relatively underrepresented compared to dominant classes, however, with approximately 1,500 training samples, it was initially expected to provide sufficient information for the model to learn meaningful patterns. Despite this its performance remains poor with only an F1-score at 0.82.

Considering the attack’s nature, might explain why the model is failing it.

A Bot attack typically operates in multiple phases. The first is the infection phase, often done through phishing, or malicious download. It occurs without any visible indications, and the user remains unaware that their machine is under the control of an attacker, effectively turning it into a “zombie” machine. The second is Command-and-Control (C2) communication where the infected machine establishes contact with an external server controlled by the attacker to receive instructions and periodically report its status. And finally using the botnet to launch an attack such as DDoS attacks, spam distribution, or data exfiltration. CIC-IDS2017 represents the second phase.

The confusion matrix in Figure 5.3 shows that bot attack is being misclassified 351 times as benign. This is due to the fact that, in this phase, the attacker

intentionally mimics benign traffic patterns in order to remain undetected, making it difficult for the model to distinguish between malicious and normal behavior. Moreover, since the dataset represents each flow independently, it fails to capture the temporal nature of bot behavior, particularly periodic communication patterns across sessions.

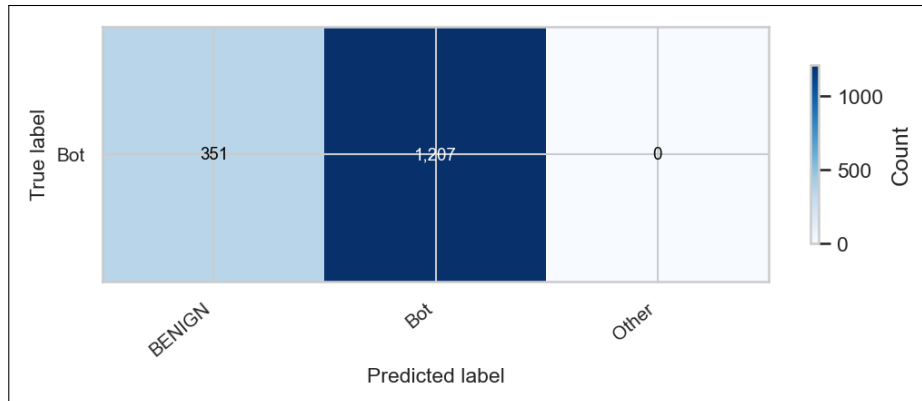


FIGURE 5.3 – Bot attack baseline confusion matrix

The baseline results show that underperforming classes do not fail for a single unified reason. Instead, two different patterns emerge. On one hand, classes such as Heartbleed and Infiltration appear to be primarily affected by extreme sample scarcity. On the other hand, Bot and Web Attack subclasses exhibit behavior that suggests additional limitations related to the nature of the data representation itself.

Despite this distinction, all these classes are still, to varying degrees, underrepresented compared to dominant classes. For this reason, it remains reasonable to investigate whether standard imbalance mitigation techniques can improve their performance before drawing definitive conclusions about the nature of these limitations.

5.4.3 Iterative Experiments

The class imbalance problem can be addressed at two levels with either an algorithm-level approach or a data-level approach :

- **Algorithm-level: Class Weights**

XGBoost is a gradient boosting model that learns by minimizing a loss function, for every wrongly predicted sample, it multiplies the loss associated with it by its assigned weight which is one by default, meaning the sample count is what primarily drives the loss function.

In highly imbalanced datasets such as CIC-IDS2017, the model naturally optimizes for the majority class because of their higher sample count.

In XGBoost, sample weights are used to adjust the importance of each individual training instance in the loss function, effectively penalizing the model more heavily for misclassifying samples. By assigning higher weights to the minority class,

this forces the algorithm to focus on correctly predicting those rare instances in subsequent boosting rounds.

This technique is a standard way to handle imbalanced datasets without modifying the training data.

- **Data-level: Data augmentation**

SMOTE (Synthetic Minority Oversampling Technique) generates new synthetic samples rather than duplicating existing ones. SMOTE works by identifying the k -nearest neighbors of an instance then selects one at random.

It then generates a new synthetic point using linear interpolation; it computes the distance between the two points, multiplies the difference by a random number between 0 and 1 and adds this value to the original sample creating the artificial new sample.

Although SMOTE becomes risky when applied to classes with very low sample counts, as the synthetic samples are created from a very limited neighborhood, which reduces diversity and can lead to unrealistic decision boundaries. This may result in generating synthetic points that do not accurately represent the real characteristics of that class, and can introduce noise.

In this case Random over sampling (ROS) becomes a better resort. It works by duplicating existing samples. It introduces no new information but ensures the class appears more frequently during training.

That said, we approach SMOTE with deliberate caution. Network intrusion data is sensitive. A synthetic sample generated by interpolating between two real flow records is not guaranteed to faithfully capture the attack pattern, which presents a real risk. It may introduce noise that disrupts the model's learned decision boundaries rather than reinforcing them. Given the critical nature of network intrusion detection, we favor XGBoost's class weighting over data augmentation, as it offers a simpler and safer mechanism that achieves a similar effect without modifying the training data. Nonetheless, we still run sampling experiments to evaluate whether the data-level approach provides any measurable gain.

We define a fixed set of experiments. If a new configuration involves altering the training set, the same experiments are rerun to ensure consistent evaluation:

1. Class weights
2. Sampling approaches (ROS and SMOTE)
3. SMOTE combined with weights

Following baseline training, we attempt to improve the performance of weaker-performing classes, namely the Web Attack subclasses, Heartbleed, Infiltration, and Bot, through these series of experiments while monitoring the evaluation metrics.

Under class weighting (balanced), Infiltration improves to an F1-score of 0.8685 and Heartbleed reaches a perfect score of 1.0. However, Bot performance deteriorates, while Web Attack classes show no measurable improvement.

Alternative Weight Formulas: A second weighting strategy is explored using two custom formulations. The first assigns weights inversely proportional to the square of the baseline F1-score, amplifying attention on the worst-performing classes disproportionately more. The second is a hybrid formula that combines a frequency-based term, the square root of the inverse class frequency, with an F1-based penalty, the inverse of the baseline F1-score, so classes that are both rare and poorly predicted receive the highest combined weight. Both formulas are normalized by their mean to keep training numerically stable. However, neither outperforms the standard balanced weighting; no improvement is observed for the target classes, and this configuration is dropped from all subsequent experiment sets.

Sampling (SMOTE + Random Oversampling):

Sampling is applied only within the training folds during cross-validation. The resampling is fit exclusively on each training fold, never on the validation fold, to prevent any form of data leakage from the synthetic samples into the evaluation.

In SMOTE, the parameter k defines the number of nearest neighbors used to generate synthetic samples. We set $k = 5$, the default value, as it ensures a stable neighborhood structure for synthetic sample generation. This choice directly influences the minimum viable class size required for SMOTE to operate correctly.

Based on this constraint, we apply SMOTE only to classes with more than 50 training samples and ROS on the ones below. This threshold is not arbitrary, SMOTE requires at least $k + 1$ samples in the minority class to identify valid neighbors. Below that count, the neighborhood becomes degenerate; the algorithm would interpolate between the same few points repeatedly, producing synthetic data that is essentially noise rather than new information.

The resampling targets are set as follows: SMOTE is used to increase Bot samples from 390 to 3,000, Web Attack – Brute Force from 294 to 3,000, and Web Attack – XSS from 522 to 2,000. Random oversampling is applied to Web Attack – SQL Injection from 17 to 300, Heartbleed from 9 to 300, and Infiltration from 29 to 300.

Under this configuration, Web Attack classes show no improvement. While Infiltration drops slightly compared to the weights configuration with F1-score = 0.8218, and Heartbleed remains at 1.0.

When combining sampling with class weighting, Bot performance drops further, whereas Infiltration recovers to F1-score = 0.9055. Web Attack classes remain unchanged.

The consistent finding across all four configurations is that Web Attack performance does not respond to any rebalancing approach. This confirms that this is not a class imbalance problem, it is a feature representation problem.

Relative to the baseline, Bot performance consistently drops under these configurations, which confirms that this is a dataset limitation.

Web Attack Class Merging

As established, no combination of weighting or sampling improves Web Attack performance. The confusion matrix shows persistent mutual misclassification between XSS and Brute Force across every experiment. Their flow-level signatures are too similar to

separate with the available features.

We therefore decide to merge the three Web Attack subclasses Brute Force, SQL Injection and XSS into a single unified Web Attack class.

Because this modifies the training set, all four experimental configurations are re-run from scratch.

The merged Web Attack class immediately achieves $F1\text{-score} = 0.987$. However, Infiltration drops from 0.7867 to 0.7267 and Heartbleed drops from 0.9333 to 0.7333 relative to the original baseline, while Bot remains stable at 0.83.

When class weights are introduced, Heartbleed returns to $F1\text{-score} = 1.0$. Infiltration recovers to 0.8685, matching its pre-merge weighted performance. Web Attack holds at 0.98.

Under sampling alone, Infiltration reaches only 0.74, Heartbleed remains at 1.0. Web Attack stays at 0.98.

Finally, combining sampling with class weights yields similar behavior, heartbleed and Web Attack stays at 1.0 and 0.98 respectively. However, infiltration returns to 0.83.

Across all four configurations, Web Attack consistently sits at approximately 0.98 regardless of whether weights or sampling are added. This confirms that merging alone is the decisive intervention; no additional rebalancing is necessary.

Heartbleed

Heartbleed warrants a closer look. At baseline, Heartbleed achieves an $F1\text{-score}$ of 0.93, yet under every subsequent configuration, weights, sampling and a mix of both, it reliably reaches 1.0.

With only 9 training samples and 2 retained in the test set, cross-validation folds see approximately 2 Heartbleed samples per validation split. At that scale, the $F1\text{-score}$ can take only a handful of discrete values, 0 when both are wrong, 0.67 in case only one is wrong, and a perfect 1.0 score means the model simply predicts both samples correctly.

More importantly, all resampling and rebalancing techniques are applied exclusively to the training folds and therefore do not affect the test distribution. As a result, regardless of whether weights or sampling are used, the final evaluation is always based on the same two test samples. This makes it hard to meaningfully assess the impact of these strategies on Heartbleed’s generalization performance.

This initially motivates an ablation experiment in which Heartbleed is temporarily removed to assess its impact on the model’s behavior.

Heartbleed ablation results

Following this change, the complete set of experiments is rerun. However, the results are unexpected: Infiltration, which previously showed significant improvement under the merged Web Attack configuration with class weighting and Heartbleed included, drops when Heartbleed is removed while keeping the same merge and weighting setup.

To verify whether this dependency is real or an artifact, we run a systematic comparison across three settings: pre-merge, merge with Heartbleed, and merge without

Heartbleed.

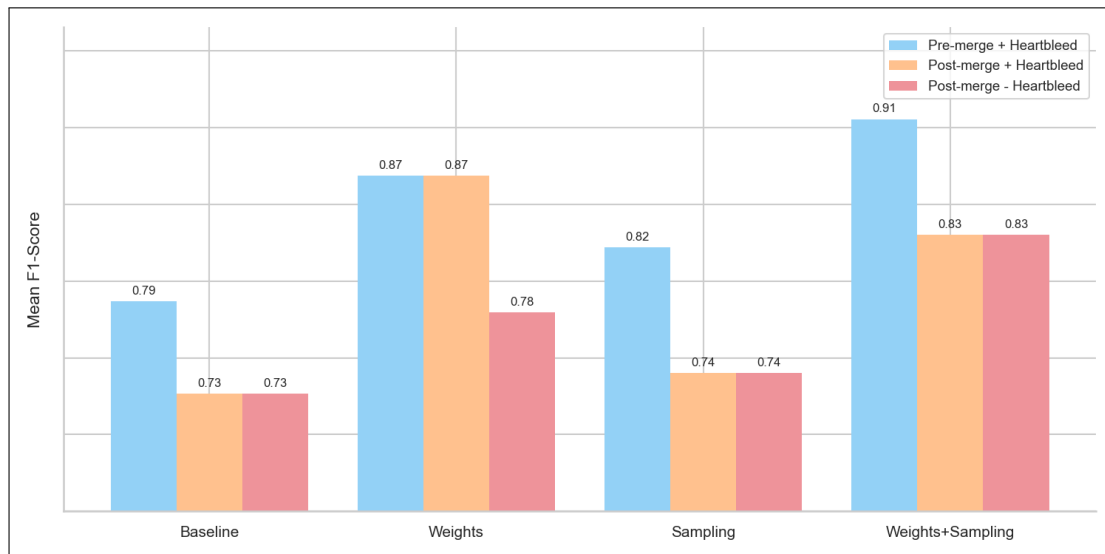


FIGURE 5.4 – Infiltration F1-score comparison across configurations

As shown in Figure 5.4, the Infiltration F1-score varies across the three configurations.

Merging Web Attacks consistently reduces Infiltration F1-score by 0.04 to 0.08 across all four strategies. This is likely because consolidating three Web Attack subtypes into one class reshapes the global decision boundary landscape in a way that slightly disadvantages Infiltration’s region.

Dropping Heartbleed at baseline has zero effect on Infiltration, with identical scores ($0.7267 = 0.7267$).

Dropping Heartbleed under weighting reduces Infiltration F1-score by up to 0.089.

To further investigate this behavior, we construct a simplified 2D representation of the problem by training a separate XGBoost model using only two selected features. In reality, the original model operates in a high-dimensional feature space composed of 46 features, making direct visualization of its decision boundaries impossible. Reducing the problem to two dimensions allows the learned feature space partitioning to be visualized and interpreted more clearly.

To select these features, we first apply SHAP (SHapley Additive exPlanations) analysis independently on both Heartbleed and Infiltration. As shown in the SHAP value distribution figures, the two features contributing most significantly to the model’s decisions for both classes are Total Length of Bwd Packets and Fwd Packet Length Max.

In the unweighted configuration with Heartbleed, as shown in Figure, the feature space is entirely dominated by the BENIGN class. Heartbleed samples cluster in the bottom-left corner but receive no dedicated boundary region. The boundary structure is flat and undifferentiated.

A similar behavior is observed in the unweighted configuration without Heartbleed, presented in Figure, the boundary is visually identical to the previous configuration. Without weighting, Heartbleed’s presence or absence has no effect on the optimization landscape.

In contrast, the weighted configuration with Heartbleed, illustrated in Figure, the boundary structure changes substantially. A dedicated region is carved out around the Heartbleed samples along the Fwd Packet Length Max axis, and a series of sharp horizontal cuts appear across the feature space. These cuts reshape the decision surface in a way that incidentally improves the separation of Infiltration samples.

When Heartbleed is removed under weighting, as shown in Figure, the horizontal structure disappears entirely and the boundary reverts to the flat BENIGN-dominated surface observed in the unweighted cases.

This behavior can be explained by the optimization mechanism of XGBoost. Under class weighting, Heartbleed receives a gradient scaling factor of approximately $19,000\times$ relative to the BENIGN class. Each of its 9 training samples exerts as much optimization pressure on the model as a substantial fraction of the majority class flows. The model, responding to this extreme gradient signal, prioritizes correctly resolving the Heartbleed decision boundary.

This extreme weighting has an indirect structural effect on Infiltration. In XGBoost, each tree is built by recursively partitioning the feature space, and each split is chosen to minimize the weighted gradient loss across all classes simultaneously. When Heartbleed receives $19,000\times$ weight, its samples impose disproportionate pressure on how certain boundaries are drawn in the shared feature subspace.

Some of these boundaries, though oriented toward separating Heartbleed from BENIGN, incidentally create a favorable partition for Infiltration samples as well, because the two classes occupy adjacent regions of feature space, the cut drawn to isolate Heartbleed simultaneously pulls Infiltration away from BENIGN. Removing Heartbleed under weighting relaxes that boundary, and Infiltration loses the incidental separation from BENIGN.

To conclude, Heartbleed and Infiltration have no structural relationship in the data. The dependency is induced entirely by the extreme weight assigned to Heartbleed.

The ablation results demonstrate that removing or amplifying Heartbleed affects specific aspects of the model’s behavior; under class weighting, its strong gradient signal contributes to sharpening nearby decision boundaries in the feature space, which indirectly benefits Infiltration performance.

This observation led to a reassessment of the initial interpretation of metric limitation.

As established, while the model achieves a perfect F1-score of 1.0, this is simply because it is evaluated on only two samples, which limits the metric to a small set of discrete outcomes, rendering evaluation unreliable. As a result, performance on this class cannot be reliably assessed from the evaluation set alone.

However, this limitation does not imply that the model fails to learn the class during training. The training process still incorporates Heartbleed samples, allowing the model to adjust its internal decision boundaries accordingly, even if this cannot be properly reflected in the test evaluation.

In light of the ablation findings, we decide to retain Heartbleed.

5.4.4 Final Model Configuration

The final model configuration carried forward into hyperparameter tuning consists of Web Attack subclasses merged into one class, Heartbleed retained, and the use of balanced class weights. Thus achieves a Macro F1-score at 0.98, with Infiltration F1-score = 0.8236, and Web Attack F1-score = 0.9896.

5.4.5 Hyper parameter tuning

5.4.6 Adding thresholds

5.4.7 Final model training results

5.5 Conclusion

Chapter 6

Realization

6.1 Introduction

6.2 Development Tools and Technologies

6.3 Performance Analysis and Evaluation Metrics

6.4 User Interfaces

6.5 Integration

6.6 Conclusion

General Conclusion

Webography

- [1] <https://www.snort.org/>. Accessed on: April 6, 2026.
- [2] <https://suricata.io/>. Accessed on: April 6, 2026.
- [3] <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/>. Accessed on: April 2, 2026.

Bibliography

- [1] Chris Jackson. *Network Security Auditing*. 800 East 96th Street, Indianapolis, IN 46240 USA: Cisco Press, 2010. ISBN: 978-1-58705-352-8.